Breakout Game

The Brief: What It Is, How It Works, What Is Wrong With It

Portfolio explainer: high-level summary

1 The project in one paragraph

`breakout-game` is a clone of the 1976 arcade game Breakout: a paddle, a bouncing ball, a wall of coloured blocks, three lives, and a persistent high score. It is 475 lines of Python across five modules and it uses **nothing but the standard library**. The point of the project is what it refuses to use: there is no game engine, so the frame loop, the collision detection, the state machine and the difficulty curve are all written by hand on top of `turtle`, a children's drawing module that was never meant to carry a game.

Why that constraint is the whole story

An engine like Pygame or Godot hands you a render loop, sprites and collision tests already solved. Removing it means writing each of them yourself, which is the exercise. Everything interesting in this codebase, and every bug in it, lives in the parts an engine would have hidden.

2 Architecture

`main.py` owns the loop and the game state. The other four modules are passive: they only act when the loop calls a method on them, and none of them knows a frame exists.

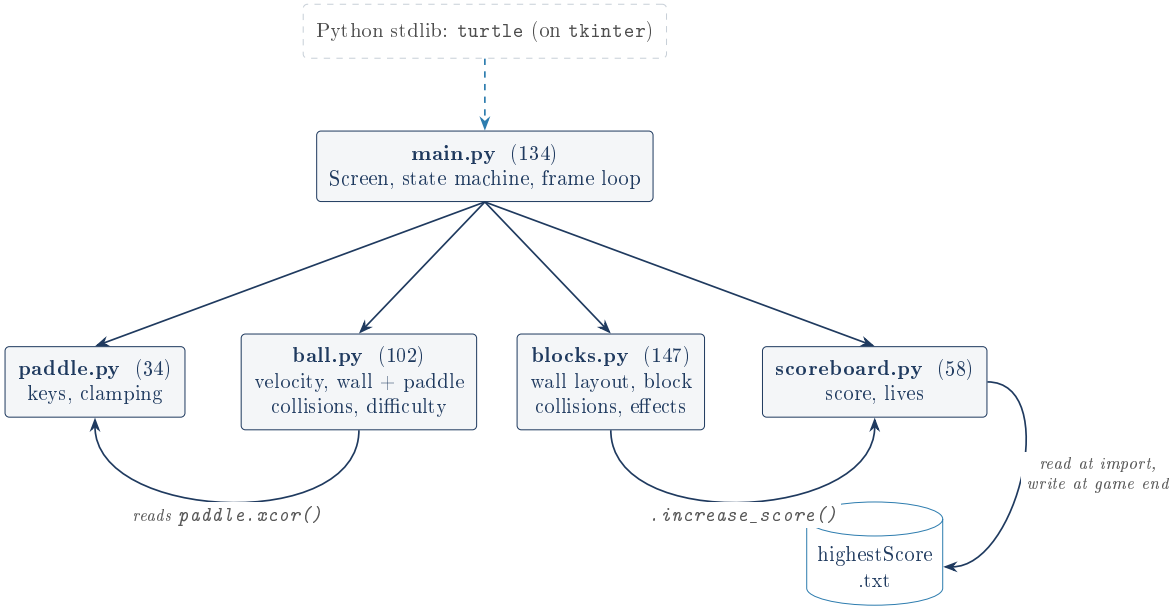

Figure 1: Five modules, line counts measured. The split is by *collision problem*: ball versus wall, ball versus paddle and ball versus block are three different shapes of maths, and each lives with the object that owns it.

3 How a frame works

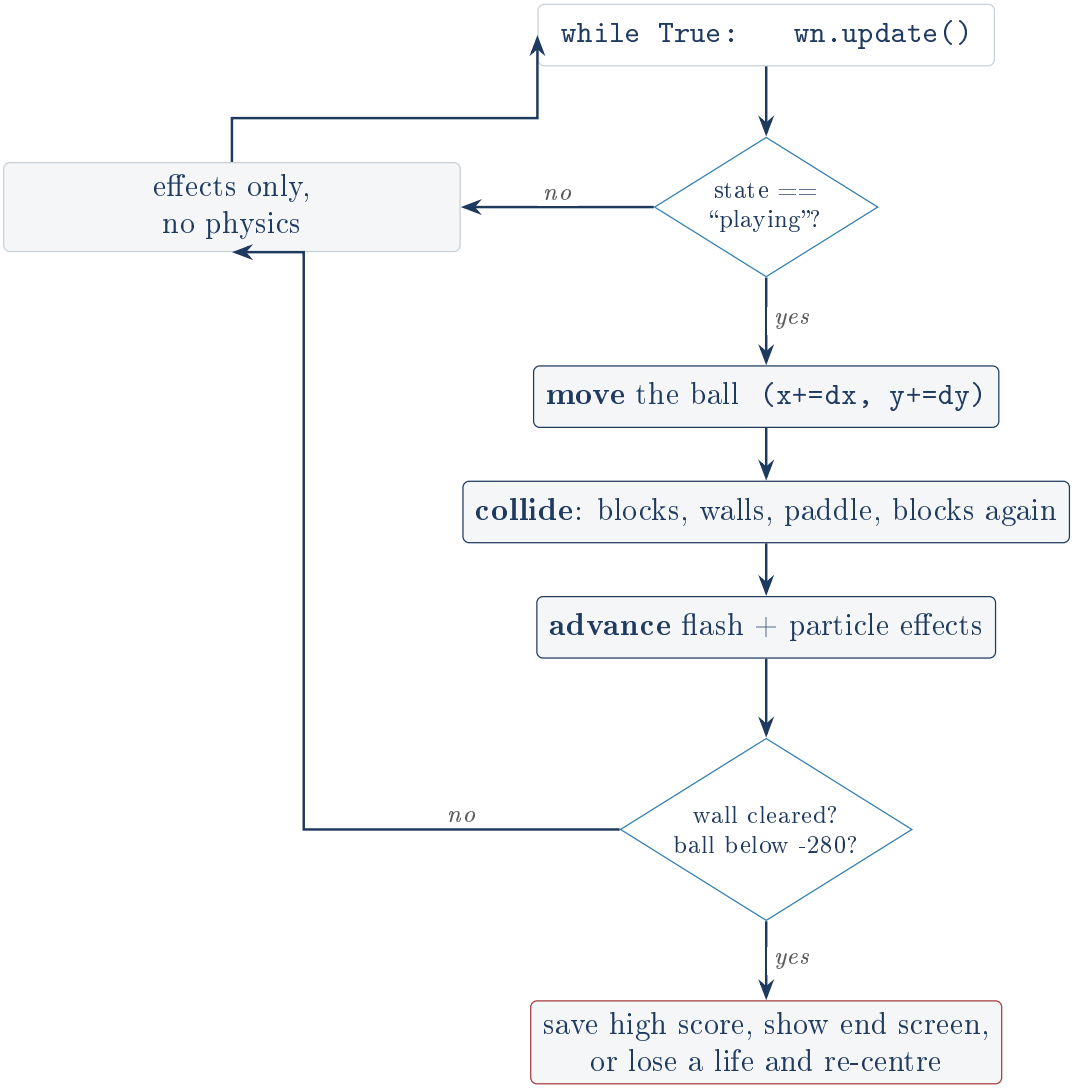

Figure 2: One frame of `main.py`. Physics runs only in the `playing` state; the other states still spin the loop so the window stays responsive to a keypress.

The one line that makes it a game

`wn.tracer(0)` switches off turtle's redraw-after-every-command behaviour, and the loop calls `wn.update()` exactly once per pass. That converts turtle from "draw each change instantly and slowly" into proper frame-based rendering: move everything invisibly, then present one complete image. It is the single non-obvious idea in the project.

3.1 State machine

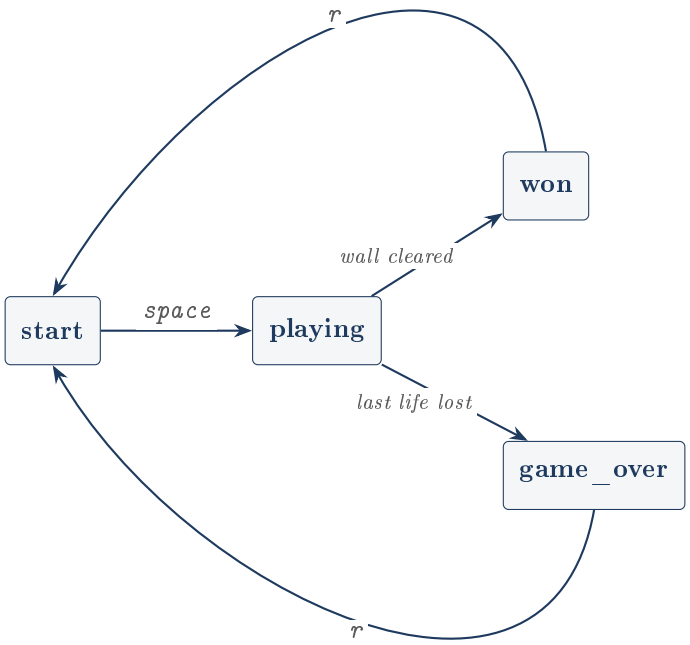

Figure 3: Four states held in one string variable. Both key callbacks are guarded, so `r` is a no-op mid-rally and `space` is a no-op on the end screen.

4 The three collision rules

Collision	Test	Response
Ball vs. wall	`|x| > 390` or `|y| > 290`	clamp back onto the boundary, then flip that axis. The clamp matters: flipping alone leaves the ball outside the wall, where it vibrates forever.
Ball vs. paddle	`y < -240` *and* within 80 units of the paddle's centre	clamp to `y = -240`, flip `dy`, count the hit. Directional on purpose: a ball moving upward passes through, which is what stops it sticking to the paddle.
Ball vs. block	`block.distance(ball) < 35`	score, remove, flash white, burst, flip `dy`. Turtle has no rectangle intersection test, so a fixed radius stands in for every block's true hitbox.

5 The rest of the design

- **Wall generation** (`blocks.py`) packs randomly-sized blocks right to left from $x = 400$, wraps down 20 units when the next will not fit, and stops when it runs out of vertical room. Each row's colour comes from its y position, so the wall reads as bands rather than a speckle.
- **Difficulty ramp**: every 4th paddle return multiplies both velocity components by 1.08. Multiplying rather than adding preserves direction, which is why it is written that way.
- **Two resets**, and the distinction is a real design decision: losing a life keeps the current speed (a setback), while restarting after game over drops back to base speed.
- **Break feedback**: a broken block flashes white for 3 frames while five particle turtles fly outward on random angles, driven by a frame countdown rather than any physics.
- **Persistence**: the high score is read from `highestScore.txt` when `scoreboard.py` is imported, and written whenever a game ends.

6 Honest assessment

Every finding below was confirmed by **running** the project's real modules, not by reading them. The Deep Dive has the evidence and the traces.

The five real defects

1. **A ghost block scores a free point on frame one of every game.** When the wall runs out of room, the last block is left at its default position, the origin, and appended to `all_blocks` anyway; `make_all_blocks()` hides it but never removes it. The ball also starts at the origin, so it collides immediately: the player gets a free point, the ball launches *downward* instead of up, and a steel-blue particle burst fires from the middle of the screen. Every symptom looks like a feature if you have not read the code.
2. **A first run starts the high score at 1.** `score = open(...).write("0")` assigns `write()`'s *return value*, which is the number of characters written. The README claims this branch starts the score at 0.
3. **The bottom wall is unreachable.** It bounces at `y < -290`, but the life-lost check fires at `y < -280` and the ball moves at most 2.6 units per frame. The README says the ball bounces off all four walls; it bounces off three.
4. **The double collision check does nothing.** `main.py` calls `check_collision_ball()` twice per frame, apparently against tunnelling, but both calls test position rather than the swept path, and a 2.6-unit step against a 35-unit radius cannot tunnel anyway. It can, however, destroy two blocks in one frame and flip `dy` twice, cancelling the bounce out.
5. **Two of the seven row colours are unreachable.** The geometry allows exactly five rows, so `'medium purple'` never appears and `'steel blue'` only ever colours the ghost block. The comment about cycling colours describes a case that cannot happen.

The design gap that matters most

The paddle is a perfect mirror: `dy` flips and `dx` is never touched, so *where* the ball hits the paddle has no effect. In real Breakout, angling the ball off the paddle's edge is the entire skill of the game. Here the player cannot aim, which means there is no strategy to play, only survival. Deriving `dx` from the strike offset is a handful of lines and is the highest-value change available to the project.

Related, and cheaper: there is no frame-rate limiter and no delta-time scaling, so the game literally runs faster on a faster computer, and the tuned constants mean nothing in absolute terms.

Documentation drift

The README quotes "279 lines" (measured: 475), says the game imports `time` (nothing does; the real imports are `turtle`, `random` and `math`), and describes the missing-file high score as 0 (it is 1). The README's judgement is sound, and its "What I'd do differently" independently identifies three of the genuine weaknesses. The drift is in the specifics.

6.1 What it gets right

- The module split earns itself: each collision rule is a different problem and lives with the object that owns it.
- `tracer(0)` plus a manual `update()` is the correct, non-obvious way to get frame-based rendering out of turtle.
- Clamp-then-flip shows the author hit the stuck-ball bug and fixed it properly, and the directional paddle test shows the same about the sticky-paddle bug.
- `for entry in self.fading[:]` shows real awareness of the mutation-during-iteration hazard, though `check_collision_ball()` only avoids the same hazard by returning early, which is correct by accident.
- Catching `Terminator` so closing the window exits cleanly, rather than dumping a traceback, is a detail most projects this size skip.

The summary

A competent exercise that does what it set out to do: a working game loop, a state machine and three kinds of hand-rolled collision detection, with a file structure the author can justify. It also ships an invisible block that scores a free point, a high score that starts at 1, three walls where the README promises four, and a paddle that cannot be aimed with. All are small fixes, and each is a better story told first than discovered by an interviewer.

7 Glossary

Game engine
A library that solves the hard parts of a game for you: render loop, sprites, collision, input, physics. This project deliberately uses none.

`turtle`
Python's standard-library drawing module, built on `tkinter`. A cursor you move around a window. Not a game library: no sprites, no collision detection, no frame timing.

Frame
One still image of the game. A game draws one, moves everything slightly, draws the next, forever.

Game loop
The `while True` that runs until the player quits. Each pass is one frame: move, collide, draw.

Velocity / `dx, dy`
Speed plus direction, stored as horizontal and vertical change per frame. Moving is addition; bouncing is multiplying one of them by -1.

Hitbox
An invisible rectangle used for collision maths instead of an object's drawn pixels. This project has none: it substitutes a fixed-radius circle for every block.

Tunnelling
When a fast object jumps clean over a thin one between two frames, so a position-based collision test never sees the overlap. Impossible at this project's speeds, which is why the defence against it is wasted work.

State machine
A design where the program is always in exactly one named state and moves between them on defined events. Here: one string, four states, two key events.

Persistence
Keeping data after the process exits, by writing it to a file. The one persisted value here is the high score.

`Terminator`
The exception `turtle` raises when its window is closed. Caught in `main.py` so the process exits quietly.